# ECM3422: Computability and Complexity

**730002704**

No AI was used to produce this assignment.

## 1. Simple Solver

For my simple solver I first implemented classes for an abstract 'expression', a literal, and the two types of clauses. These make the solver implementation simpler by providing `evaluate` and `get_vars` functions to evaluate the expression (that is, determine its truth value given some assignment) and get the set of variables used in the expression respectively. These classes are also used in the optimised solver.

The solver works by recursively calling `solve_with_assignment`, which takes an expression and an assignment. If this function is called with a complete assignment of variables then it evaluates the expression given the assignment and returns its truth value. If called with an incomplete assignment it picks a variable to assign (the 'choice variable') and recursively calls with this variable assigned true and then false. If neither assignment leads to the expression being satisfied, the solve function returns false.

In the worst case this algorithm will have to assign every variable twice, first to true and then to false. With $n$ variables this leads to a complexity of $O(2^n)$. A formula which is only satisfied when all variables are assigned false would lead to this scenario.

Some simple optimisations that could be made might include early-exiting from an assignment, for instance if one side of a conjunction is known to be false. This would decrease the complexity for some formulae, but still retain the same worst case.

*(231 words)*

## 2. Optimised Solver

The advanced solver implements the DPLL algorithm. This algorithm requires the input formula be in Conjunctive Normal Form (CNF), that is, a conjunction of disjunctions such as:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee x_5)$$

This is done by distributing any disjunctions, using the identity:

$$a \vee (x \wedge y) \equiv (a \vee x) \wedge (b \vee y)$$

Once it is guaranteed that the formula is in this form, it is converted into a 2-dimensional list, where the inner lists are of disjunctions (called clauses) and the outer list is a conjunction over these clauses.

Note advantages that CNF provides, namely that we now know that a clause is true iff at least one literal in it is true, and the formula is true iff every clause is true. This leads to the two rules of DPLL:

- Unit Propogation: If we have a clause where all but one literal is false, the remaining literal must be assigned true for the clause to be true.
- Pure Literal Elimination: If a variable occurs with only one sign throughout the formula (is always true or always false), then we should assign it to the value making it true.

If neither of these rules can be applied, we instead select and propogate an arbritrary variable.

In my implementation, when a clause becomes true it is removed from the formula as we no longer need to consider it. When a literal becomes false it is removed for the same reason. This process is called unit propogation and is carried out whenever an assignment is made (in fact, it is the only function that needs to be performed to 'assign' a literal).

In the worst case DPLL performs the same as the simple method: $O(2^n)$ for $n$ variables. This would apply, for instance, on a formula consisting only of clauses containing two unique (within the entire formula) literals.

*(291 words)*

# 3. Comparison

The file `test.py` contains code to generate random formulae (controlled by a few parameters) and then measure how long it takes the two implementations to solve them[1]. Running the file gives output such as:

```
── Results from 5,000 tests ──
Simple avg:   194.1µs
Advanced avg: 21.9µs
CNF Conversion avg: 1350.9µs

Took ~1s
Max Depth: 5, Density: 0.8, Num Vars: 5
```

And produces three graphs (Figure 1, Figure 2, Figure 3), showing the performance of the solvers as the three generation parameters are varied.

The generation paramters are controlled to produce a variety of formulae, ranging from single literals to large trees. They are:

- **Max Depth**: The maximum nesting that the formula can have. When the formula is being recursively generated, this is the parameter that decreases until, at `max_depth=0`, the function always returns a single literal.
- **Density**: The probability that a generated expression is a literal as oposed to a conjunction or disjunction. At `density=1`, the generated formula will always be the largest binary tree that can be generated given the set depth.
- **Num Vars**: The maximum number of variables that the formula will contain. At the start of generating the formula, a list of `num_vars` variables is created, and this is sampled from when a variable is needed.

From the basic text output of running a large number of tests we can make some observations:

1. The simple implementation is considerably (10x in the above example) slower than the advanced implmenentation.
2. The time taken to convert an arbritrary formula into CNF is considerably slower than the time for either solver.

The first observation is as expected, and the relationship between the two is best explored in the produced graphs. The second observation is, I think, more interesting. Although little effort was put into optimising the function that converts an expression into CNF[2], it is still suprising quite how slow it is. If the input formula was known to be CNF this step could of course be removed.

If 'density' and 'depth' are assumed to be correlated to the size of the formula, Figure 1 and Figure 2 show quite clealy that:

1. The solve time of both implementations increases exponentially with formula size.
2. The DPLL (advanced) solver scales significantly slower than the simple solver.

Increasing the number of different variables in the formula affects the simple solver much more than the advanced solver. This is likely because the advanced solver takes much better advantage of the extra variables through pure literal elimination and unit propogation. The simple solver, of course, is highly dependent on the number of variables, given its brute force approach.

One advantage of the simple solver is that it actually finds a satisfying assignment, whereas implementing this in the advanced solver would require some changes to the implementation.

[1] Whilst it does this, the responses from the two implementations are compared to assure they are the same.

[2] `to_cnf` in `advanced.py`.

*(473 words)*

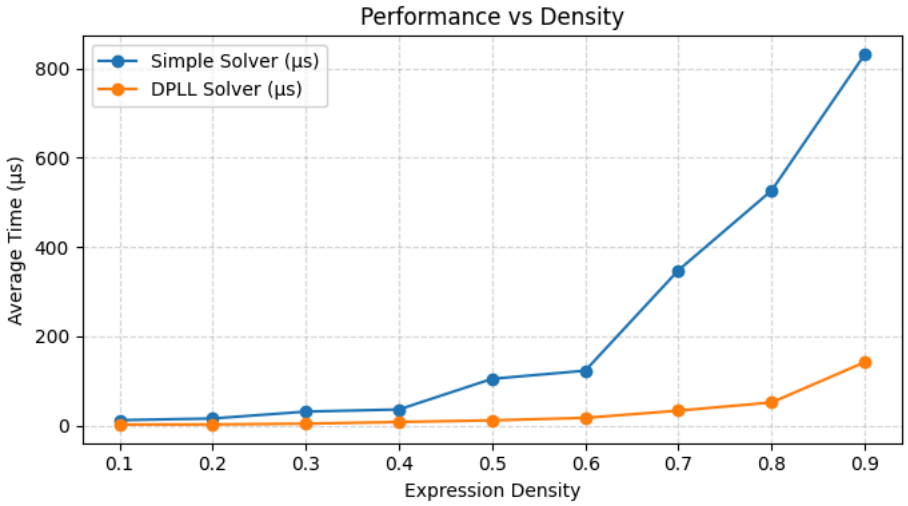


Figure 1: Variation in the solvers' performances with '`density`' (n=100). `depth=5, num_vars=5`.

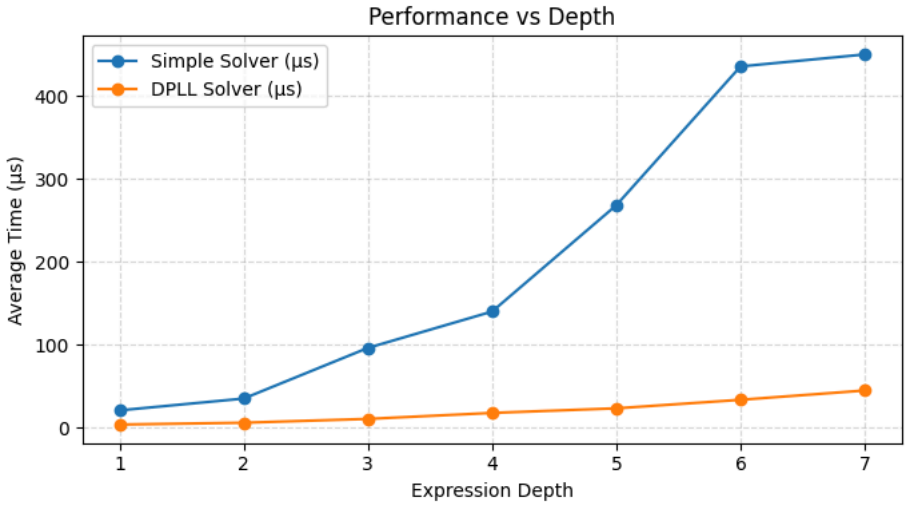


Figure 2: Variation in the solvers' performances with '`max_depth`' (n=100). `density=0.7, num_vars=5`.

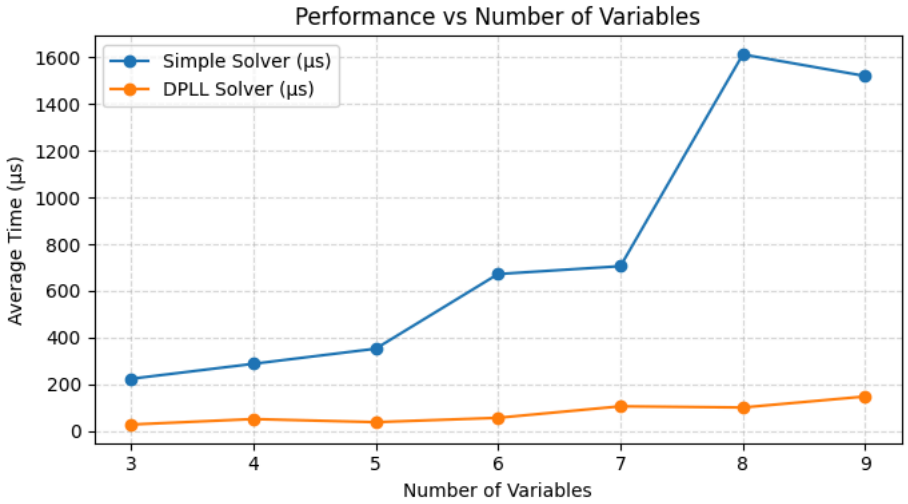


Figure 3: Variation in the solvers' performances with '`num_vars`' (n=100). `depth=5, density=0.7`.